Lab 03 — UDP & DNS Traffic Analysis

Protocols: UDP (User Datagram Protocol) + DNS (Domain Name System)
Tool: Wireshark on Kali Linux
RFC: UDP — RFC 768 | DNS — RFC 1035
Difficulty: Beginner-Intermediate
Time Taken: ~2 hours

Objective

Capture DNS queries and responses over UDP to understand how domain name resolution works at the packet level. DNS is one of the most critical — and most abused — protocols in cybersecurity. Analyzing raw DNS traffic reveals what websites users are visiting, enables DNS cache poisoning understanding, and shows why DNS-over-HTTPS (DoH) exists.

Protocol Background

UDP — The Connectionless Alternative to TCP

UDP (User Datagram Protocol) operates at **Layer 4 (Transport Layer)** alongside TCP, but with a key difference: **no connection, no acknowledgment, no guarantee of delivery.**

Feature	TCP	UDP
Connection required	Yes (handshake)	No
Reliable delivery	Guaranteed	Best-effort
Ordering	In-order	No ordering
Speed	Slower	Faster
Header size	20 bytes	8 bytes
Use cases	HTTP, SSH, FTP	DNS, VoIP, Gaming, Video

UDP's minimal header:

Source Port (16 bits) | Destination Port (16 bits)

Length (16 bits)| Checksum (16 bits)

DATA (variable)

DNS — The Internet's Phone Book

DNS translates human-readable domain names to IP addresses. Without DNS, every website would require memorizing an IP address.

You type: www.google.com

DNS resolves: www.google.com → 142.250.192.132

Browser connects to: 142.250.192.132:443

DNS uses UDP port 53 for queries (typically). For large responses (>512 bytes), it falls back to TCP port 53.

DNS Resolution Chain

Browser → Recursive Resolver (ISP/8.8.8.8)

→ Root Nameserver (.)

→ TLD Nameserver (.com)

→ Authoritative Nameserver (google.com)

← IP Address returned

← IP cached + returned to browser

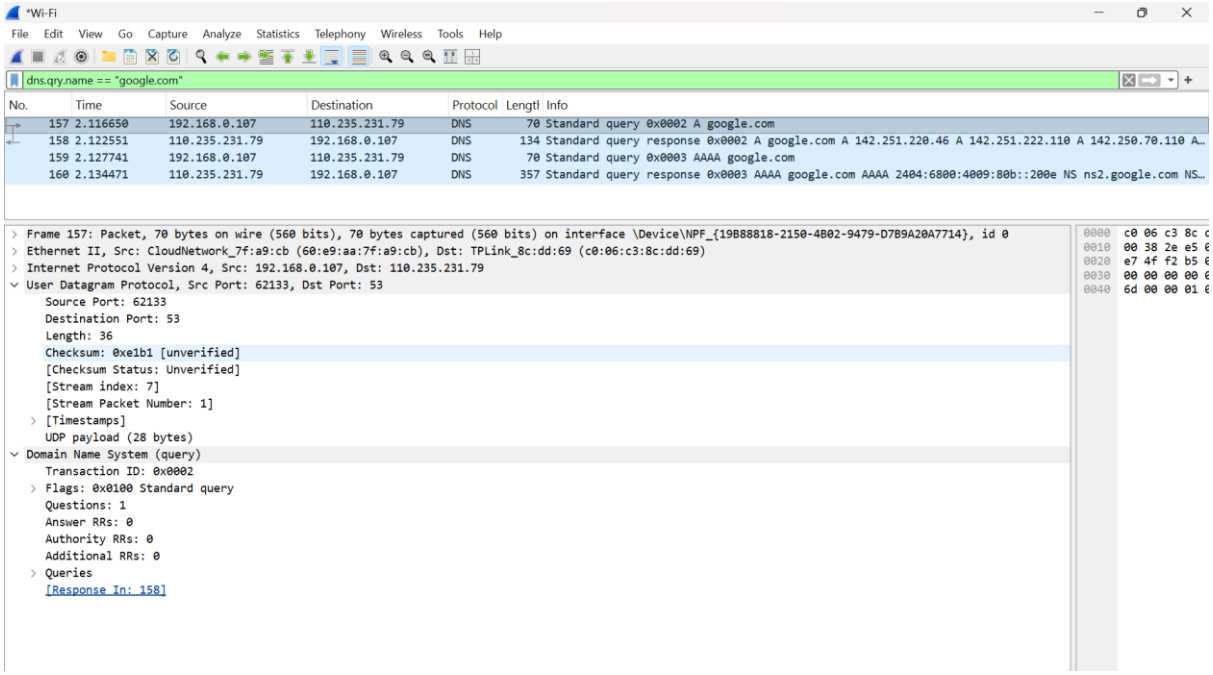
DNS Record Types

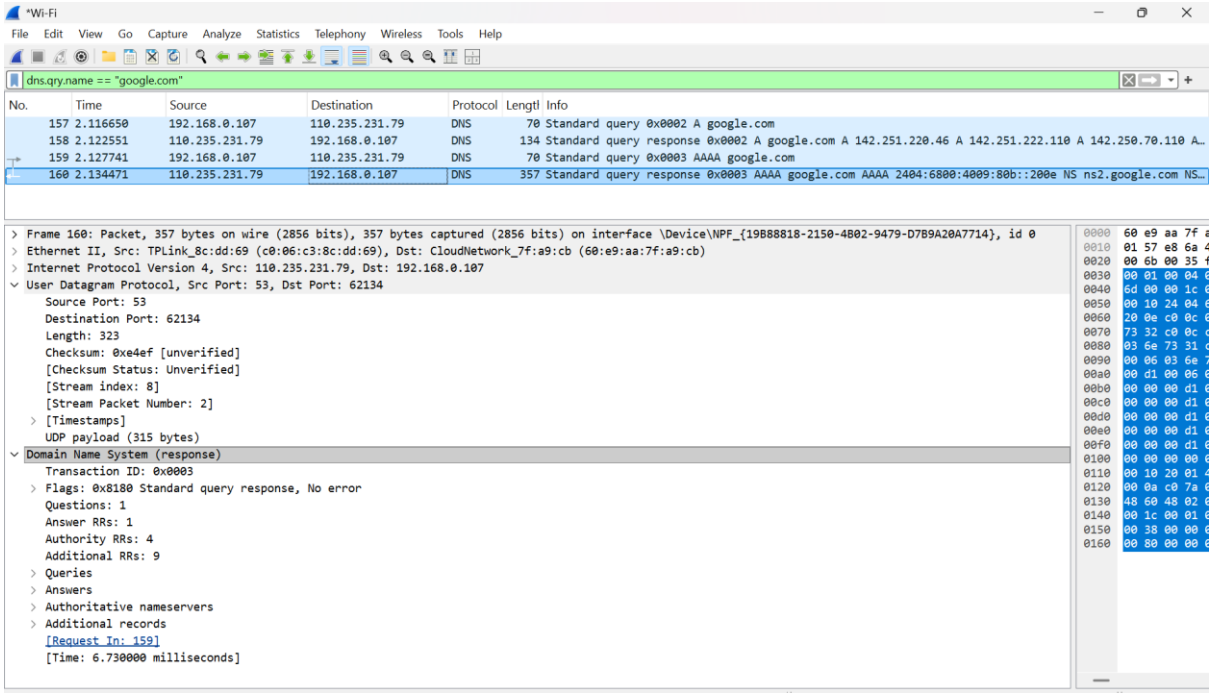
Type	Purpose	Example
A	IPv4 address	google.com → 142.x.x.x
AAAA	IPv6 address	google.com → 2607:f8b0::...
MX	Mail server	gmail.com → mail server
CNAME	Alias/redirect	www → canonical name
TXT	Metadata / SPF / verification	v=spf1 include:...
NS	Authoritative nameserver	who owns this zone
PTR	Reverse DNS (IP → name)	1.1.168.192 → host.local

What I Captured

```
C:\Users\DELL>nslookup google.com
Server:  hyd-tdc-bngs-25
Address:  110.235.231.79

Non-authoritative answer:
Name:    google.com
Addresses:  2404:6800:4009:80b::200e
          142.251.220.46
          142.251.222.110
          142.250.70.110
          142.250.70.78


C:\Users\DELL>nslookup google.com
Server:  hyd-tdc-bngs-25
Address:  110.235.231.79

Non-authoritative answer:
Name:    google.com
Addresses:  2404:6800:4009:80b::200e
          142.251.220.46
          142.251.222.110
          142.250.70.110
          142.250.70.78
```

```
C:\Users\DELL>nslookup -type=MX gmail.com
Server:  hyd-tdc-bngs-25
Address:  110.235.231.79

Non-authoritative answer:
gmail.com       MX preference = 20, mail exchanger = alt2.gmail-smtp-in.l.google.com
gmail.com       MX preference = 10, mail exchanger = alt1.gmail-smtp-in.l.google.com
gmail.com       MX preference = 5, mail exchanger = gmail-smtp-in.l.google.com
gmail.com       MX preference = 30, mail exchanger = alt3.gmail-smtp-in.l.google.com
gmail.com       MX preference = 40, mail exchanger = alt4.gmail-smtp-in.l.google.com

gmail.com       nameserver = ns4.google.com
gmail.com       nameserver = ns2.google.com
gmail.com       nameserver = ns1.google.com
gmail.com       nameserver = ns3.google.com
gmail-smtp-in.l.google.com      internet address = 142.250.4.26
ns2.google.com  internet address = 216.239.34.10
ns1.google.com  internet address = 216.239.32.10
ns3.google.com  internet address = 216.239.36.10
ns4.google.com  internet address = 216.239.38.10
gmail-smtp-in.l.google.com      AAAA IPv6 address = 2404:6800:4003:c06::1a
ns2.google.com  AAAA IPv6 address = 2001:4860:4802:34::a
ns1.google.com  AAAA IPv6 address = 2001:4860:4802:32::a
ns3.google.com  AAAA IPv6 address = 2001:4860:4802:36::a
ns4.google.com  AAAA IPv6 address = 2001:4860:4802:38::a
(root)  ??? unknown type 41 ???
```





Lab Setup

Command run on Kali:

nslookup google.com # Simple DNS query

nslookup -type=MX google.com # Query mail records

dig google.com # Detailed DNS lookup

```
dig google.com +trace     # Full resolution chain
```

With Wireshark running and filter udp.port == 53 applied, every DNS query and its response is captured.

Wireshark Filters Used

```
# All DNS traffic
dns

# DNS over UDP only
udp.port == 53

# DNS queries only (QR bit = 0)
dns.flags.response == 0

# DNS responses only (QR bit = 1)
dns.flags.response == 1

# DNS A record queries
dns.qry.type == 1

# DNS for specific domain
dns.qry.name == "google.com"

# Failed DNS (non-zero response code)
dns.flags.rcode != 0

# Large DNS responses (possible tunneling)
dns && udp.length > 512
```

Screenshots & Findings

Screenshot 1 — DNS UDP Packet for nslookup google.com

What this shows:

- Wireshark dissects the DNS query sent by nslookup google.com
- **Source port:** Random ephemeral port (e.g., 51234) — client-assigned
- **Destination port:** 53 — DNS server
- **Transaction ID:** Random 16-bit value — matches query to response
- **Flags: 0x0100** — Standard query, recursion desired
- **Query:** Type A, Class IN (Internet), Name = "google.com"

Full DNS header breakdown visible:

Transaction ID: 0x[random]

Flags: 0x0100

  QR: 0 (Query)

  Opcode: 0 (Standard query)

  RD: 1 (Recursion Desired)

Questions: 1

Answer RRs: 0 ← empty on query

Authority RRs: 0

Additional RRs: 0

Query:

  Name: google.com

  Type: A (Host Address)

  Class: IN (Internet)

Screenshot 2 — DNS Response from Google

What this shows:

- DNS server responds with the A record for google.com
- **Same Transaction ID** as the query — this is how the response is matched

- **Flags: 0x8180** — Response, recursion available
- **Answer section:** Contains the IP address mapping

Full response breakdown:

Transaction ID: 0x[same as query]

Flags: 0x8180

QR: 1 (Response)

AA: 0 (Not Authoritative — this came from cache/recursive resolver)

RD: 1 (Recursion Desired)

RA: 1 (Recursion Available)

RCODE: 0 (No error)

Questions: 1

Answer RRs: 1

Answers:

Name: google.com

Type: A

Class: IN

TTL: 299 (seconds until cache expires)

Address: 142.250.x.x ← THE IP ADDRESS

Critical observation: This entire exchange — including the domain being visited — is **sent in plaintext over UDP**. Anyone on the network can see exactly which domains you're querying.

DNS in the Real World — What One Capture Reveals

In a typical 5-minute browsing session, DNS traffic reveals:

- Every website visited (full domain names)
- Frequency of visits (repeated queries)
- Browser behavior (prefetch DNS requests for links you haven't clicked yet)
- Internal infrastructure (internal domain names and their IPs)

Example of what a DNS capture might look like:

Query → facebook.com

Query → instagram.com

Query → api.facebook.com

Query → graph.facebook.com

Query → static.xx.fbcdn.net

An attacker watching your DNS traffic knows exactly what you're doing online — **without decrypting a single HTTPS packet.**

Security Implications

1. DNS Eavesdropping (Passive Surveillance)

Since DNS uses UDP without encryption, any attacker on the same network (coffee shop Wi-Fi, ISP) can:

- See every domain you query
- Build a profile of your browsing behavior
- Identify security tools you use (queries for antivirus update servers, VPN endpoints)

```
# Attacker captures DNS with tcpdump

tcpdump -i eth0 -nn port 53

# Every domain query prints in plaintext
```

2. DNS Cache Poisoning (BGP Hijack / Kaminsky Attack)

The Kaminsky Attack (2008) exploited DNS's weak randomization:

1. Attacker floods recursive resolver with forged DNS responses
2. If attacker's forged response arrives before the real one, it gets cached
3. All users relying on that resolver now get attacker's IP for the domain
4. Users are redirected to attacker's fake website

How Wireshark shows this: Multiple response packets to the same Transaction ID — the real server and the attacker racing to respond.

3. DNS Tunneling (Data Exfiltration)

Tools like **iodine**, **dnscat2**, and **DNScat** encode data in DNS queries to create a covert C2 channel:

Normal: query → google.com

Tunneled: query → c3RvbGVuZGF0YQ==.evil.com (stolen data encoded in subdomain)

Detection:

```
# Unusually long DNS query names

dns.qry.name.len > 50


# High frequency queries to same domain
```

Many unique subdomains of same domain

4. DNS Spoofing / On-Path Attack

On a local network, an attacker using ARP spoofing (see Lab 6) can intercept DNS queries and send forged responses:

Victim queries: bank.com → ?

Attacker intercepts and responds: bank.com → [Attacker's phishing IP]

Victim's browser connects to: Attacker's fake bank website

5. DNS Amplification (DDoS)

Send DNS queries with **victim's IP spoofed as source**. DNS response (much larger than query) gets sent to the victim — amplification factor of 50-100x:

Attacker: 50 byte query → DNS server (src=Victim IP)

DNS server: 3000 byte response → Victim IP

Amplification: 60x

Defender Perspective

DNS-over-HTTPS (DoH) / DNS-over-TLS (DoT)

The modern solution to DNS eavesdropping:

DoH: DNS queries sent inside HTTPS to port 443

DoT: DNS queries sent inside TLS to port 853

Both **encrypt DNS queries**, preventing eavesdropping. Configurable in browsers (Firefox, Chrome) and OS-level resolvers.

DNSSEC — Signing DNS Records

DNSSEC adds cryptographic signatures to DNS records, preventing cache poisoning:

- Each DNS response carries a cryptographic signature
- Resolvers verify the signature against the zone's public key
- Forged responses fail signature verification and are discarded

In Wireshark, DNSSEC responses include: RRSIG, DNSKEY, DS records — look for these in DNS answer sections.

Rate Limiting and Monitoring

Detect DNS tunneling with pattern matching

Alert if single host queries > 100 unique domains/minute

Alert if DNS payload > 512 bytes

Alert if subdomain length > 50 characters

Key Takeaways

1. **DNS is entirely plaintext** — ISPs, attackers, and network admins can see every domain you visit
2. **UDP's connectionless nature** makes DNS responses easy to forge (no handshake to verify)
3. **Transaction ID** is the only thing linking a query to a response — if predictable, it's exploitable
4. **DNS tunneling bypasses firewalls** — outbound port 53 is almost never blocked
5. **DoH/DoT** are the modern standard — but they move the trust from ISP to DNS provider (Cloudflare/Google)
6. **TTL values in DNS** control caching — short TTLs mean more queries, more exposure

---